RetailEdge Inc. — AWS Architecture

Hardened Three-Tier Design

This document is the AWS architecture for RetailEdge Inc.'s e-commerce platform. It incorporates the full set of required security, reliability, and operational changes on top of the original three-tier design: a fully private application tier reached with no NAT Gateway, WAF and CloudFront edge protections, a hardened Redis caching layer, least-privilege IAM, GitHub OIDC and Blue/Green CI/CD, remote Terraform state, and end-to-end logging and monitoring.

1. Architecture Overview

The solution remains organized into three layers, now hardened end-to-end:

- **Web / Edge Tier:** Route 53, CloudFront, and AWS WAF associated with the CloudFront distribution. CloudFront uses two origins and separate routing behaviors: the default behavior serves static web content from S3, while /api/* requests are routed through a CloudFront VPC Origin to the internal ALB.
- **Application Tier:** Internal Application Load Balancer (ALB) in private application subnets, followed by auto-scaled EC2 instances in private application subnets. The ALB handles dynamic application/API traffic only and is reachable only through the CloudFront VPC Origin. EC2 instances have no public IPs and require no NAT Gateway; required AWS services are accessed through VPC Endpoints.
- **Data Tier:** RDS MySQL (Multi-AZ), ElastiCache Redis (Multi-AZ, encrypted), and S3. RDS and Redis remain private and are reachable only from the application tier, while the web S3 bucket serves static content through CloudFront using Origin Access Control (OAC).

2. High-Level Diagram

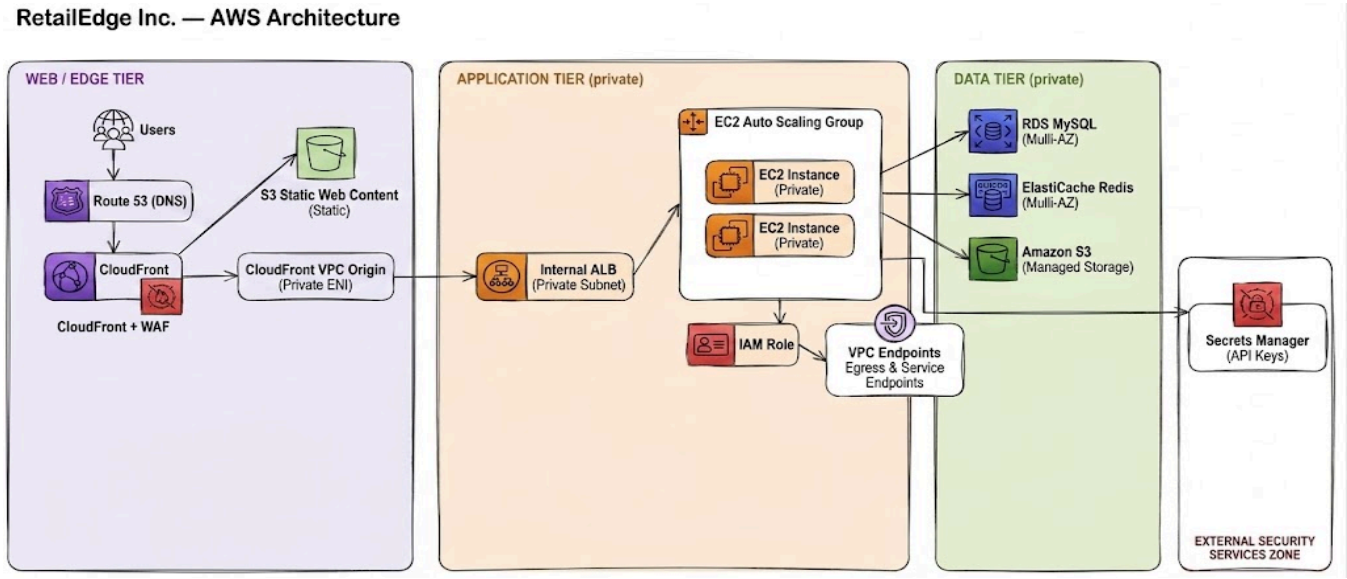

Figure 1 — High-level AWS architecture

3. Web / Edge Tier

- Amazon Route 53 provides DNS resolution for the application domain.
- CloudFront is the public entry point and CDN layer for the application. The CloudFront distribution is associated with AWS WAF for edge-level protection.
- CloudFront uses two origins and separate routing behaviors:
  - Default/static behavior: requests that do not match /api/* are routed to the private S3 web bucket and cached using the AWS Managed-CachingOptimized cache policy.
  - Dynamic API behavior: /api/* requests are routed through the CloudFront VPC Origin to the internal ALB. Caching is disabled using the AWS Managed-CachingDisabled policy.
- The React web application is built by the CI/CD pipeline and synchronized to the private S3 web bucket using aws s3 sync. The S3 bucket therefore contains the latest production web build.
- The S3 web bucket is private and is accessed by CloudFront using Origin Access Control (OAC) with SigV4 signing. Direct public access to the bucket is blocked.
- After a successful web deployment, CloudFront serves the updated static assets from S3. Cache invalidation can be performed for updated assets when required.
- The internal ALB handles dynamic API traffic only and is not directly exposed to the public internet.
- HTTPS is used from the viewer to CloudFront and from CloudFront to the VPC Origin/ALB.
- ACM provides the certificate required for HTTPS on the internal ALB.
- The internal ALB listens on HTTPS port 443. No public HTTP listener is required because the ALB is internal and CloudFront communicates with it over HTTPS.
- The ALB Security Group allows inbound HTTPS traffic only from the AWS-managed CloudFront origin-facing prefix list.
- ALB deletion protection is enabled in production to prevent accidental deletion of the main ALB.

4. Application Tier

- The application tier handles dynamic API traffic only. Static web assets are served directly from the S3 origin through CloudFront.
- The application tier consists of an internal Application Load Balancer followed by an Auto Scaling Group of EC2 instances in private subnets.
- The internal ALB is reachable only through the CloudFront VPC Origin. It is not directly accessible from the public internet.
- EC2 Auto Scaling Group instances sit in private subnets, have no public IPs, and are not directly reachable from the internet. Only the ALB can reach the application instances on TCP port 8080.
- The base AMI contains only infrastructure dependencies: Docker and SSM Agent, together with the required OS configuration. It does not contain DB/Redis passwords, AWS access keys, application secrets, or application releases.
- Application releases are decoupled from the AMI. The Docker image is built by GitHub Actions and pushed to Amazon ECR. The deployment process then instructs the running EC2 instances to pull the new image from ECR and restart the application container.
- AWS Lambda acts as the deployment orchestration function. After a successful image push, GitHub Actions invokes the Lambda function with the new ECR image tag.

- The Lambda function uses AWS Systems Manager Run Command to execute the deployment command on the application EC2 instances. The command authenticates to ECR using the EC2 instance IAM role, pulls the specified image, stops the previous container, starts the new container, and validates the application health endpoint.
- The deployment targets the application instances managed by the Auto Scaling Group so the application version can be updated without rebuilding the AMI.
- Each instance uses a least-privilege IAM Instance Role (RetailEdgeAppInstanceRole) to access ECR, Secrets Manager, and SSM. No long-lived AWS credentials are stored on the instances.
- Administration and deployment operations use AWS SSM Session Manager and Run Command. SSH port 22 is not exposed publicly.

5. Data Tier

5.1 Relational Database — Amazon RDS MySQL

- Multi-AZ for high availability; remains private with no public access.
- Automated backups are configured explicitly for production with a backup retention period of 7 days.
- Production database deletions retain a final snapshot unless there is an explicit reason to skip it.
- The application uses a MySQL connection pool with a bounded maximum number of connections instead of opening a new connection per request, to avoid overwhelming RDS.
- RDS is protected by a dedicated Security Group that allows MySQL traffic on TCP port 3306 only from the Application Security Group

5.2 Caching — Amazon ElastiCache for Redis

- Deployment: Provisioned · Cluster Mode: Disabled · Multi-AZ: Enabled · Automatic Failover: Enabled · Replica: at least 1. Start with a small node and scale after measuring real usage.
- Encryption at rest and encryption in transit (TLS) are both enabled; if Redis auth is enabled, the auth token is stored in Secrets Manager (retailedge/redis) rather than in code.
- The Redis client is configured in RedisConfig.js; the endpoint and auth token are obtained at runtime, never hardcoded.
- The Redis client is created once at application startup and reused — a new connection is not opened per request.

5.3 Cache-Aside Pattern (TransactionService.js)

- Read path: check Redis first. On a cache hit, return the cached value directly with no database query. On a cache miss, read from RDS, return the data, and populate Redis with a configurable TTL (e.g. CACHE_TTL=60).
- Write path (POST / PUT-UPDATE / DELETE): after the RDS write succeeds, the affected Redis cache keys are invalidated (deleted) to prevent stale reads.
- Failure handling: Redis is a cache, not the source of truth. If Redis is unavailable, the application falls back to RDS directly and continues functioning, with degraded performance; the Redis client's reconnect/failover behavior is configured accordingly.

5.4 Object Storage — Amazon S3

- S3 stores the application's static web assets and serves them through the CloudFront S3 origin.
- S3 is not part of the dynamic API request path; dynamic requests are routed through the CloudFront VPC Origin to the internal ALB.
- S3 buckets use SSE-S3 (AES256) encryption and Block Public Access.
- Versioning is enabled on the application buckets where appropriate.
- The web bucket is private and allows s3:GetObject access only to the CloudFront service principal through an S3 bucket policy restricted by the CloudFront distribution ARN.
- The application assets bucket uses a least-privilege bucket policy that grants the EC2 application role only the required S3 permissions.

6. Network Architecture

A single VPC (10.0.0.0/16) spans two Availability Zones. The application tier does not rely on a NAT Gateway for outbound access — private EC2 instances reach required AWS services entirely through VPC Endpoints. The CloudFront-to-S3 static content path is handled through the CloudFront S3 origin, while dynamic API traffic enters the VPC through the CloudFront VPC Origin and reaches the internal ALB.

Layer	Availability Zone A	Availability Zone B
Public	10.0.1.0/24	10.0.2.0/24
Application	10.0.11.0/24	10.0.12.0/24
Database	10.0.21.0/24	10.0.22.0/24

The public subnets are provisioned as part of the VPC structure but are not used by the internal ALB or EC2 application instances. The ALB and EC2 instances are deployed in the private application subnets.

6.1 VPC Endpoints (replacing the NAT Gateway)

- **Interface Endpoints:** ECR API, ECR DKR, Secrets Manager, SSM, SSMMessages, and EC2Messages.
- **Interface Endpoint ENIs** are deployed in the private application subnets and protected by a dedicated Endpoint Security Group.
- The **S3 Gateway VPC Endpoint** is associated with the application route table to provide private S3 access from the application tier when required.
- CloudFront accesses the S3 static-content origin separately through the CloudFront/S3 origin configuration using OAC.

7. Security Architecture

Security follows least privilege throughout: resources communicate only over the ports and with the components they explicitly require. Static content is served directly from the private S3 origin through CloudFront, while dynamic API traffic is routed through the CloudFront VPC Origin to the internal ALB.

Security Group	Port	Allowed Source	Purpose
ALB SG	443 (HTTPS)	AWS-managed CloudFront origin-facing prefix list	Allow CloudFront VPC Origin to reach the internal ALB
Application SG	8080 (HTTP)	ALB SG only	App tier traffic

Security Group	Port	Allowed Source	Purpose
RDS SG	3306 (MySQL)	Application SG only	Database access
Redis SG	6379	Application SG only	Cache access
Endpoint SG	443	Application SG only	Private access to ECR, Secrets Manager, SSM, and other required AWS services

7.1 IAM vs. Security Groups — Clarification

The EC2 instances do not need an IAM role "for RDS" or "for Redis." These are two distinct control planes:

- **IAM Role:** controls which AWS APIs the EC2 instance may call (EC2 → IAM Role → Secrets Manager: GetSecretValue).
- **Security Groups:** control which network connections are allowed (EC2 → TCP 3306 → RDS, and EC2 → TCP 6379 → Redis).

In short, IAM governs AWS API permissions, while Security Groups govern network-level connectivity.

7.2 IAM Role for EC2 (RetailEdgeAppInstanceRole)

- **Secrets Manager:** `secretsmanager:GetSecretValue`, scoped only to the required DB and Redis secrets.
- **ECR:** `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`, and `ecr:BatchCheckLayerAvailability`.
- **SSM**: `AmazonSSMManagedInstanceCore` attached to allow Systems Manager management and Run Command execution.
- No AWS access keys are placed on EC2 instances. The EC2 instance role provides temporary AWS credentials through the instance metadata service.
- The EC2 role does not require CodeDeploy permissions because application deployment is performed through AWS Systems Manager.

7.3 Secrets Manager

- The database secret follows the environment-aware naming convention: <project>-<environment>/db.
- The Redis secret follows the environment-aware naming convention: <project>-<environment>/redis.
- The database secret contains the database connection information, including host, username, password, database name, and port.
- The Redis secret contains the Redis connection information, including host, port, authentication token, and TLS configuration.
- Runtime retrieval follows: EC2 → IAM Role → Secrets Manager → DB/Redis credentials → application memory only.
- Secrets are never stored in GitHub, the Docker image, the Dockerfile, the AMI, source code, or a static .env baked into the image, and are never printed to logs.

8. Container and Deployment Architecture

The application artifact remains separate from the EC2 server image, so application releases never require a new AMI. Deployment is orchestrated through GitHub Actions, Amazon ECR, AWS Lambda, and AWS Systems Manager.

- The EC2 AMI contains only the operating system, Docker, SSM Agent, and required infrastructure dependencies.
- GitHub Actions builds the application Docker image and pushes it to the Amazon ECR repository.
- Each application release is tagged with a unique image tag, such as the Git commit SHA, so deployments reference an immutable application version rather than relying on a mutable `latest` tag.
- After the image is successfully pushed to ECR, GitHub Actions invokes the deployment Lambda function and passes the ECR image URI/tag.
- The Lambda deployment function uses AWS Systems Manager Run Command to target the application EC2 instances managed by the Auto Scaling Group.
- On each instance, the deployment command:
    1. Authenticates Docker to Amazon ECR using the EC2 instance IAM role.
    2. Pulls the specified application image.
    3. Stops and removes the currently running application container.
    4. Starts the new container with the required runtime configuration.
    5. Retrieves application secrets at runtime from Secrets Manager.
    6. Performs a health check against the **`/health`** endpoint.
- The deployment does not store application secrets in the Docker image, AMI, GitHub repository, or deployment commands.
- If the health check succeeds, the new application version remains running.
- If deployment validation fails, the deployment process reports the failure and the previous image can be restored on the affected instances.
- Auto Scaling remains responsible for maintaining the required number of healthy EC2 instances. New instances launched by the ASG start from the base AMI and receive the application release through the deployment mechanism.

9. CI/CD Architecture

GitHub Actions authenticates to AWS through GitHub OIDC rather than long-lived access keys. The IAM trust policy is restricted to the exact repository and branch/ref, e.g. **`repo:your-org/retailedge-app:ref:refs/heads/main`**.

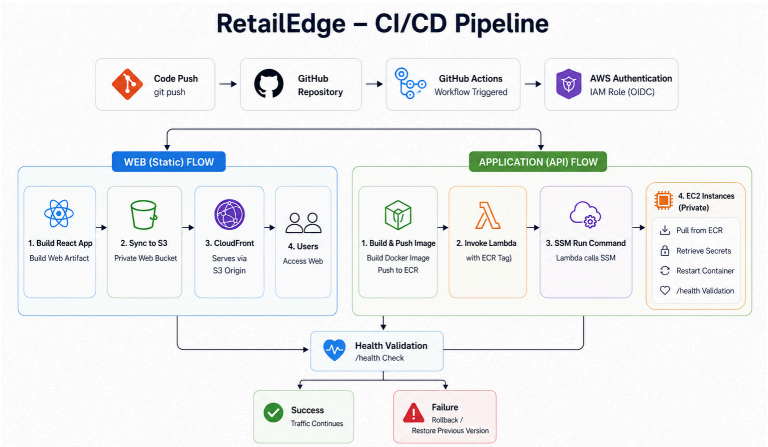

Figure 2 — CI/CD pipeline flow

The pipeline is:

Developer push → GitHub Actions → OIDC → temporary AWS credentials → tests → build Docker image → push to ECR → build React web application → sync static assets to S3 → invoke deployment Lambda → Lambda → SSM Run Command → EC2 instances pull the new ECR image → container restart → `/health` validation → production

The deployment process is divided into two application artifacts:

- **Web deployment:** GitHub Actions builds the React application and synchronizes the generated static files to the private S3 web bucket. CloudFront then serves the updated files through the S3 origin.
- **API deployment:** GitHub Actions builds the backend Docker image and pushes it to ECR using a unique release tag. The deployment Lambda is then invoked with the new image tag.
- **Deployment Lambda:** The Lambda function uses AWS Systems Manager Run Command to execute the container update on the application EC2 instances.
- **EC2 deployment:** Each targeted instance pulls the specified image from ECR, retrieves runtime secrets from Secrets Manager, starts the new container, and validates `/health`.
- **Approval:** A manual approval step can be placed before the production deployment.
- **Failure handling:** If deployment validation fails, the pipeline reports the deployment as failed and the previous application image can be restored.

GitHub Actions does not require permanent AWS access keys. OIDC provides temporary AWS credentials for the workflow, while EC2 instances use their IAM instance role for AWS service access

10. Infrastructure as Code

Terraform continues to own the infrastructure lifecycle. Shared production state is no longer local:

- Remote backend: Terraform → S3 backend, with the state bucket encrypted, blocking public access, versioned, and restricted to authorized IAM principals.
- State locking uses whichever locking mechanism the Terraform version in use supports (a dedicated DynamoDB table if using the traditional locking approach).
- Requirement in one line: remote, encrypted, protected, and locked Terraform state.

11. Monitoring, Logging, and Auditing (Updated)

Component	Key Metrics / Logs	Example Alarm / Use
ALB	Request count, target response time, 4xx, 5xx, healthy host count	P95 latency > 800ms for 5 min → SNS
EC2 / ASG	CPU, memory, network, instance & healthy-host status	Instance health alarms → SNS
RDS	CPU, connections, storage, read/write latency	CPU / connection alarms → SNS
Redis	CPU, memory, connections, hits/misses, evictions, hit ratio	Cache hit ratio alarms → SNS

Component	Key Metrics / Logs	Example Alarm / Use
ALB Access Logs	Path, status code, timing, client info → S3	Traffic investigation / troubleshooting
WAF Logging	Blocked/allowed requests, triggered rules, rate limiting	Investigate suspicious traffic, false positives
CloudTrail	Who / what action / when / which resource across AWS APIs	IAM, Secrets Manager, RDS, S3, SG, Terraform changes
Lambda Deployment	Invocations, errors, duration, throttles	Deployment failure investigation
SSM Run Command	Command status, execution failures, deployment output	Deployment troubleshooting and failure detection

12. Final Checklist

Network

- Private EC2 subnets, no public IPs
- VPC Endpoints instead of NAT (ECR API/DKR, Secrets Manager, SSM endpoints, S3 gateway)
- Dedicated Endpoint Security Group

Edge / Security

- Route 53
- CloudFront as the public entry point
- CloudFront VPC Origin pointing to the internal ALB
- AWS WAF associated with the CloudFront distribution
- Separate CloudFront cache behaviors for static and dynamic/API content
- ACM certificates
- HTTPS from viewer → CloudFront → VPC Origin → ALB
- Internal ALB with HTTPS listener on port 443
- No public ALB listener
- ALB deletion protection enabled in production

Security Groups

- CloudFront VPC Origin → ALB 443
- ALB → App 8080
- App → RDS 3306
- App → Redis 6379
- App → VPC Endpoints 443
- No public access to the ALB, application, RDS, or Redis tiers

IAM & Secrets

- Least-privilege EC2 instance role; no AWS access keys on EC2; GitHub OIDC with restricted trust policy
- DB and Redis credentials in Secrets Manager, retrieved at runtime; none in code, AMI, image, or Git

RDS & Redis

- Multi-AZ RDS, automated backups + retention + final snapshot policy, connection pooling, parameterized queries
- Redis: Multi-AZ, replica, automatic failover, encryption at rest + TLS, cache-aside, TTL, invalidation, failure fallback, connection reuse

Deployment & CI/CD

- Base AMI contains Docker and SSM Agent; no application release or secrets are baked into the AMI
- GitHub Actions authenticates to AWS through OIDC
- React application is built and synchronized to the private S3 web bucket
- CloudFront serves static web assets from S3 through OAC
- Backend Docker images are built and pushed to ECR
- ECR images use unique release tags
- Deployment Lambda orchestrates application updates
- AWS Systems Manager Run Command updates the application EC2 instances
- EC2 instances pull the specified image from ECR using their IAM instance role
- Runtime secrets are retrieved from Secrets Manager
- `/health` validation is performed after deployment
- Deployment failures are detected and the previous image can be restored
- Auto Scaling maintains the required number of healthy application instances
- No CodeDeploy dependency
- No long-lived AWS credentials on EC2 or GitHub Actions

S3, Terraform, and Observability

- S3 encryption, Block Public Access, versioning, least-privilege IAM
- Remote encrypted/versioned/locked Terraform state in S3
- ALB access logs, WAF logs, CloudTrail, CloudWatch alarms including cache hit/miss ratio